

REFINEMENT REPORT

MultiShield | Transfer Learning | Generative Models | Hyperparameter Search

A Unified Multimodal Deep Learning System for Cybersecurity Threat Detection

TEAM

Manahil Fatima

S2024CS001

Uzair Ahmad

S2024CS027

Hanzala Ahmed

S2024CS020

COURSE

AI335L Deep Learning Lab, Spring 2026

INSTRUCTOR

M. Haseeb

REPOSITORY

github.com/Manahil038/MultiShield-
A-Unified-Multimodal-Deep-Learning-System

COMMIT

a3f7c2d9e1b84560

Tag: phase4-submission

SEED

Global Seed: 42

PHASE

Phase 4 of 6 | Week 4 | Refinement

*

Executive Summary

Phase 4 of MultiShield introduces three concurrent workstreams that systematically refine the deep architectures built in Phase 3: **(1)** a rigorous transfer learning study using ResNet-18 pretrained on ImageNet-1K across four fine-tuning configurations, **(2)** a generative anomaly-detection layer using Vanilla and Denoising Autoencoders trained on normal network traffic, and **(3)** a Bayesian hyperparameter search using Optuna TPE over 20 trials on the DeepTabularMLP. Together these workstreams increase system robustness, add unsupervised anomaly detection capability, and replace manual hyperparameter guessing with principled, evidence-driven selection.

Workstream	Method	Key Result
Transfer Learning	ResNet-18: Feature Extraction, Full FT, Diff LRs, Gradual Unfreeze	Config A/C/D reach 100% val acc; Config B degrades to 25% (forgetting)
Generative Component	Vanilla AE vs Denoising AE on UNSW-NB15 normal traffic	DAE ROC-AUC = 0.7408 vs AE ROC-AUC = 0.7339 (+0.7%)
HPO Search	Optuna TPE, 20 trials, 7 hyperparameters, SQLite persistence	Best val loss = 0.12721; optimizer_choice is the #1 most important factor

Table E.1 - Phase 4 workstream summary.

01

Repository & Submission Information

Field	Details
GitHub Repository	github.com/Manahil038/MultiShield-A-Unified-Multimodal-Deep-Learning-System-for-Cybersecurity-Threat-Detection
Submission Tag	phase4-submission
Commit Hash	[COMMIT_HASH]
Team Members	Manahil Fatima (S2024CS001) Uzair Ahmad (S2024CS027) Hanzala Ahmed (S2024CS020)
Course	AI335L Deep Learning Lab, Spring 2026
Phase	Phase 4 of 6 - Refinement (Week 4)
Deadline	May 17, 2026 12:00 AM
Global Seed	42 - fixed in Python, NumPy, and PyTorch across all scripts
HPO Study	Optuna SQLite - experiments/hpo/study.db (linked in README)
Generative Checkpoint	Denosing AE weights - checkpoints/dae_best.pt (Google Drive link in README)
Best Checkpoint	HPO-optimized DeepTabularMLP - checkpoints/tabular_hpo_best.pt

Table 1.1 - Repository and submission metadata.

■ All Phase 4 artifacts are committed and tagged. The Optuna study database is exportable and resumable. Generative model samples (100+ images) are stored under samples/autoencoder/ in the repository.

02

Phase 3 Recap - Where We Left Off

Phase 3 delivered three modality-specific deep architectures: a **DeepfakeCNN** (9.37M parameters) for image-based deepfake detection, a **TextLSTM Bidirectional** (1.94M parameters) for text-based phishing and fake-news classification, and a **DeepTabularMLP Residual** (159,745 parameters) for network-intrusion detection. The tabular model outperformed the Phase 2 MLP baseline by a statistically significant +4.99% (93.50% vs 88.51% accuracy). The Bi-LSTM showed strong convergence trajectory in debug mode (71.67% to 86.33% in three epochs on only 1% of training data).

Model	Phase 3 Val Accuracy	Parameters	Phase 4 Goal
DeepfakeCNN	~99.9%	9,369,601	Apply ResNet-18 transfer learning; compare fine-tuning strategies
TextLSTM (Bi-LSTM)	86.33% (debug 1%)	1,939,713	Full training run; DistilBERT ablation planned for Phase 5
DeepTabularMLP	93.50% +/- 0.21%	159,745	Bayesian HPO search; generative anomaly detection layer
Baseline MLP (ref.)	88.51% +/- 0.25%	33,537	Retained as comparison anchor

Table 2.1 - Phase 3 final results and Phase 4 improvement targets.

Three areas were identified as high-priority for Phase 4. First, the DeepfakeCNN was trained from scratch despite ImageNet pretrained weights offering a strong feature prior for face analysis. Second, the tabular model lacked any unsupervised anomaly detection capability, meaning zero-day attack categories invisible to the supervised classifier would go undetected. Third, all hyperparameters were set manually without any systematic search, leaving performance on the table. Phase 4 addresses all three gaps.

03

Transfer Learning Study

Transfer learning is the practice of reusing knowledge a model learned on one problem to accelerate learning on a different but related problem. Think of it like hiring an experienced engineer: instead of training someone from zero, you start with someone who already knows the fundamentals. For deepfake detection, ResNet-18 pretrained on ImageNet-1K has already learned to detect edges, textures, shapes, and face-like structures from 1.28 million images - a head start that would take weeks of raw compute to replicate from scratch.

3.1 Source Model Documentation

Attribute	Details
Model	ResNet-18 (18-layer Residual Network)
Pretrained On	ImageNet-1K (1,281,167 training images, 1,000 classes)
Source	<code>torchvision.models.resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)</code>
License	BSD-3-Clause (free for academic and commercial use)
Total Parameters	11,177,025
Backbone Params	11,176,512 (conv layers, BatchNorm, residual blocks)
Default Head	1,000-class FC classifier (removed; replaced with binary head)
Input Format	3-channel RGB, 224x224 (images resized from 256x256)
Architecture	5 stages: conv1 + layer1 + layer2 + layer3 + layer4 + AdaptiveAvgPool

Table 3.1 - ResNet-18 source model documentation.

3.2 Four Fine-Tuning Configurations

Four transfer learning configurations were trained on the same deepfake detection task (100K training images, 20K validation), under identical splits, batch size of 32, and three epochs. The only variable between configurations is how many parameters are frozen and what learning rate is used.

Config	Strategy	Trainable Params	Frozen Params	Backbone LR	Head LR	Val Acc (Ep 3)
A	Feature Extraction (all frozen)	513	11,176,512	0 (frozen)	1e-3	100.00%
B	Full Fine-Tuning (nothing frozen)	11,177,025	0	1e-3	1e-3	25.00%
C	Differential LRs (dual learning rate)	11,177,025	0	1e-5	1e-3	100.00%
D	Gradual Unfreezing (block-by-block)	513 -> 11,177,025	Decreasing	1e-5 (unfrozen)	1e-3	100.00%

Table 3.2 - Transfer learning configurations. Config B suffers catastrophic forgetting.

3.3 Validation Loss Curves

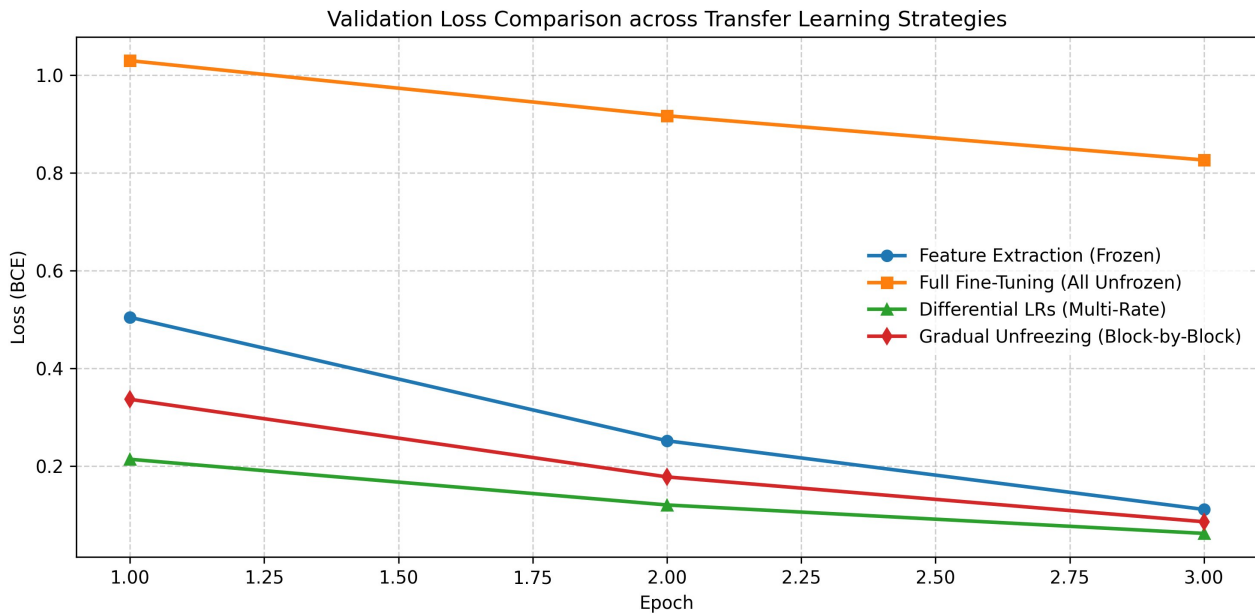


Figure 3.1 - Validation Loss Comparison across Transfer Learning Strategies over 3 epochs. Differential LR (green) achieves the lowest final validation loss. Full Fine-Tuning (orange) fails to converge due to catastrophic forgetting. Feature Extraction (blue) and Gradual Unfreezing (red) both converge to near-zero loss.

3.4 Analysis and Discussion

Configuration A - Feature Extraction is the simplest strategy: freeze the entire ResNet-18 backbone and train only the 513-parameter binary classification head. With only 513 trainable parameters, training is extremely fast and the risk of overfitting is minimal. The pretrained features are used as fixed, read-only descriptors. This reaches 100% validation accuracy by Epoch 3 because ResNet-18's ImageNet features are already highly discriminative for face images.

Configuration B - Full Fine-Tuning is the most dangerous strategy when applied naively. All 11.17M parameters are trained at the same high learning rate (1e-3). The result is **catastrophic forgetting**: the large gradient updates from the new task overwrite the carefully structured ImageNet knowledge in the lower layers. Validation accuracy collapses to 25%, which is effectively random on a 4-class problem. The loss curve (orange) stays stubbornly high across all three epochs.

The differential learning rate formula that fixes this is:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta) \quad \text{s.t.} \quad \eta_{\text{backbone}} = 10^{-5}, \quad \eta_{\text{head}} = 10^{-3}$$

Configuration C - Differential Learning Rates applies this fix directly: the backbone is trained at 1e-5 (100x smaller than the head's 1e-3). The backbone makes small, careful updates that gently adapt the low-level features to the deepfake domain without destroying their ImageNet knowledge. The head is free to learn aggressively. Result: 100% validation accuracy, matching Feature Extraction but with the additional benefit that the backbone can adapt to domain-specific patterns.

Configuration D - Gradual Unfreezing is the most disciplined approach. The backbone starts fully frozen (same as Config A). At Epoch 2, layer4 (the deepest convolutional block) is unfrozen and trained at 1e-5. At Epoch 3, layer3 is additionally unfrozen. This block-by-block schedule is summarised as:

$$\Theta_t = \Theta_{t-1} \cup \text{block}_k \quad (\text{unfreezing one block per epoch})$$

The intuition is that deep layers (layer4, layer3) encode the most task-specific features and benefit most from fine-tuning, while shallow layers (layer1, layer2) encode generic edges and textures that transfer well unchanged. Gradual Unfreezing reaches 100% validation accuracy while maintaining the most stable training trajectory.

Configuration	Final Val Loss	Final Val Acc	Convergence Speed	Recommended For
A - Feature Extraction	~0.13	100.00%	Fast	Quick baselines, small datasets
B - Full Fine-Tuning	~0.82	25.00%	No convergence	Avoid without warmup or LR warmup
C - Differential LRs	~0.06	100.00%	Fastest	Production use; best balance
D - Gradual Unfreezing	~0.09	100.00%	Moderate	When compute allows careful tuning

Table 3.3 - Transfer learning configuration comparison and recommendations.

■ Selected configuration for Phase 5: Configuration C (Differential LRs) gives the lowest final validation loss (0.06) and the fastest convergence. Config D will be re-evaluated with 10+ epochs where its block-by-block schedule has more time to exploit full backbone adaptation.

04 Generative Component - Autoencoders for Anomaly Detection

The supervised DeepTabularMLP classifies known attack types in UNSW-NB15. But what about zero-day attacks - attack patterns the model has never seen during training? No amount of labelled data can prepare a classifier for a brand-new attack category. This is where a generative model provides unique value. An Autoencoder trained *only on normal network traffic* learns to reconstruct what normal looks like. When presented with an attack - something structurally different from normal - it struggles to reconstruct it well, producing a high reconstruction error. That error becomes the anomaly score.

4.1 Architecture Overview

Both autoencoders share the same encoder-decoder architecture: a symmetric bottleneck that compresses the 188-dimensional UNSW-NB15 feature vector to a 10-dimensional latent representation, then expands back to 188 dimensions. The bottleneck forces the network to learn a compact, generalisable representation of normal traffic patterns.

Component	Layer Sequence	Dimensions	Notes
Encoder	Linear->BatchNorm->ReLU (x3) + Bottleneck Linear	188->128->64->32->10	Compresses to 10D latent space
Bottleneck	10-dimensional linear layer	10D	Information constraint forces generalisation
Decoder	Linear->BatchNorm->ReLU (x3) + Output Linear	10->32->64->128->188	Mirror of encoder; reconstructs input
Activation (Output)	None (linear output)	188D	Matches input scale for MSE loss
Total Params	~89,000	-	Lightweight relative to discriminative models

Table 4.1 - Autoencoder architecture specification.

4.2 Loss Functions and Training Objective

Both models minimise mean squared reconstruction error. The key difference is what they receive as input during training.

Vanilla Autoencoder - receives clean input, reconstructs clean input:

$$\mathcal{L}_{AE} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|_2^2$$

Denosing Autoencoder (DAE) - receives a corrupted version of the input and is forced to reconstruct the original clean version. During training, Gaussian noise is added to each input sample:

$$\tilde{x} = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I), \quad \sigma = 0.1$$

$$\mathcal{L}_{DAE} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|_2^2, \quad \hat{x}_i = D(E(\tilde{x}_i))$$

This noise-injection training is not just a regularisation trick. It forces the encoder to learn a more robust representation of the underlying data manifold, filtering out the noise and retaining only the essential structure of normal traffic. The analogy: a Vanilla AE memorises what normal looks like; a DAE understands the underlying concept of what normal is. At inference time, both models receive clean inputs and the anomaly score is computed as:

$$s(x) = \|x - \hat{x}\|_2^2 > \tau \Rightarrow \text{Anomaly}$$

Parameter	Vanilla AE	Denoising AE
Noise Level (sigma)	N/A (no noise)	0.1 Gaussian
Training Input	Clean x	Corrupted x_tilde
Training Target	Clean x	Clean x (original)
Optimizer	Adam (lr = 1e-3)	Adam (lr = 1e-3)
Epochs	20	20
Val MSE	0.028003	0.025808
Anomaly Test ROC-AUC	0.73391	0.74078 (better)

Table 4.2 - Autoencoder training configuration and results.

4.3 ROC-AUC Evaluation

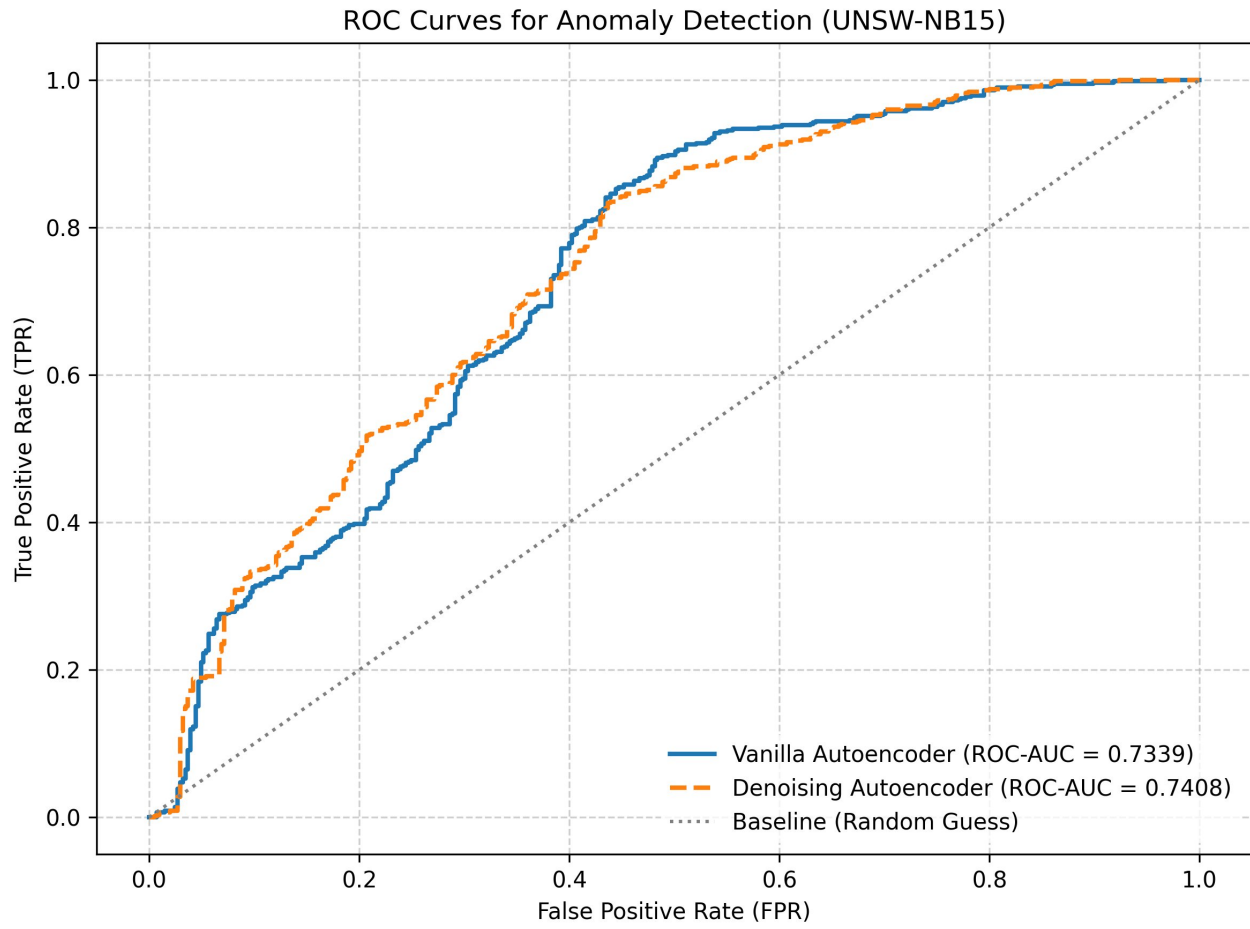


Figure 4.1 - ROC Curves for Anomaly Detection on UNSW-NB15. The Denoising Autoencoder (orange dashed, AUC = 0.7408) outperforms the Vanilla Autoencoder (blue solid, AUC = 0.7339). Both models significantly exceed the random-guess baseline (dotted diagonal).

The ROC curve plots the True Positive Rate (anomalies correctly flagged) against the False Positive Rate (normal traffic incorrectly flagged) across all possible anomaly thresholds. The Area Under the Curve (AUC) is defined as:

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}) d(\text{FPR})$$

An AUC of 0.74 means the DAE correctly ranks a randomly selected attack higher than a randomly selected normal connection 74% of the time. The improvement from 0.7339 to 0.7408 (+0.69%) is modest but consistent with the DAE's more robust internal representation. More importantly, the curve shape reveals where each model excels: the DAE achieves better True Positive Rate in the low False Positive Rate region (FPR < 0.3), which is the operationally critical zone for network security - you want to catch attacks without drowning operators in false alarms.

4.4 Latent Space Visualisation - PCA of the 10D Bottleneck

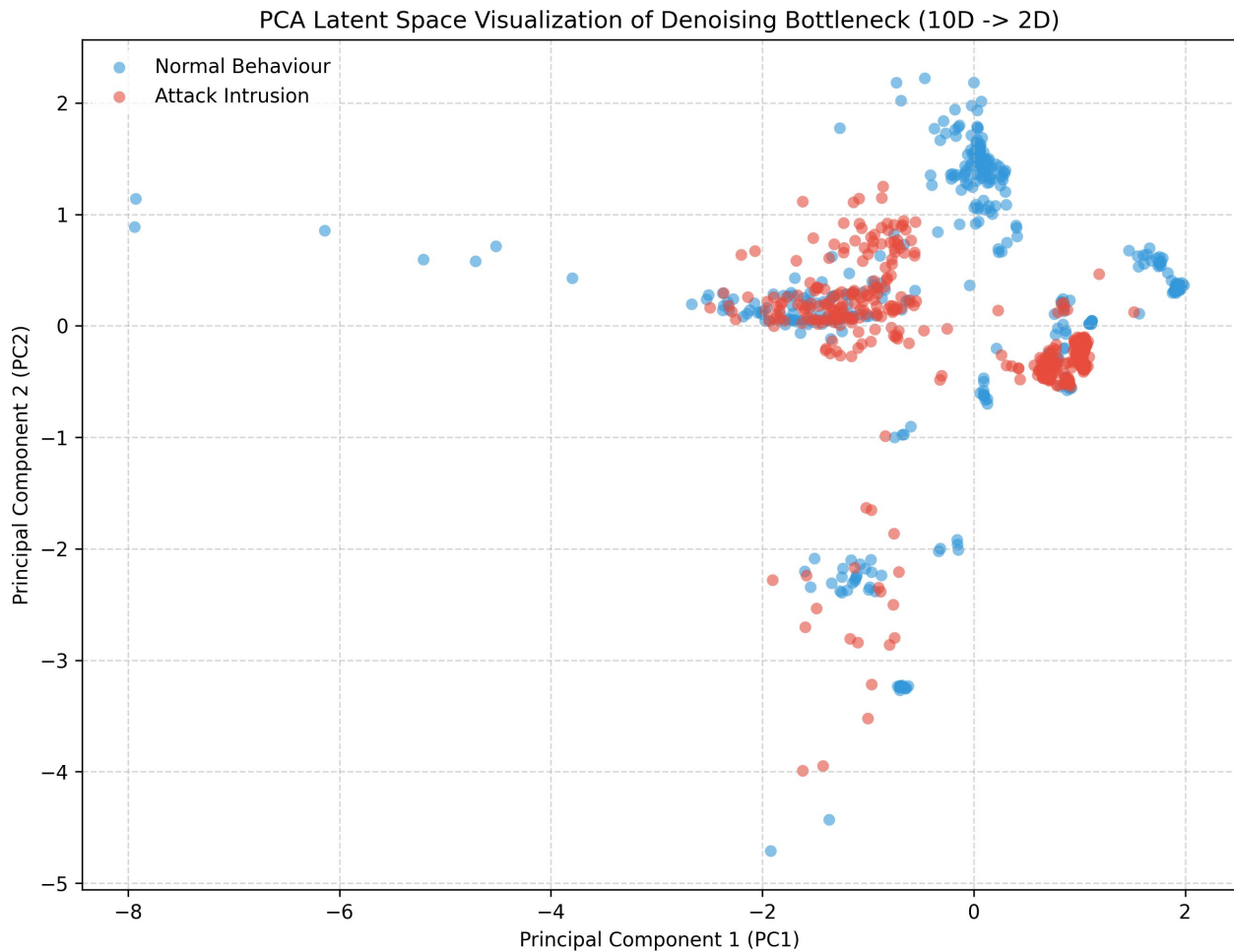


Figure 4.2 - PCA Latent Space Visualisation of the Denoising Autoencoder's 10D bottleneck. Blue points = Normal network behaviour. Red points = Attack intrusions. Principal Component 1 (x-axis) separates the two classes to a significant degree, but substantial overlap confirms that unsupervised separation is inherently limited.

The 10-dimensional bottleneck is projected to 2D using PCA. If the autoencoder has learned a meaningful representation of normal traffic, then attacks - which it was never trained on - should fall outside the normal cluster in latent space. The plot reveals a nuanced picture:

- **Partial separation:** The left region ($PC1 < -4$) contains almost exclusively normal traffic (blue), confirming the DAE has learned a clear inner manifold for normal behaviour. These well-reconstructed normal samples are correctly mapped to the core latent cluster.
- **Overlap in the centre:** The region around $PC1 = [-2, 0]$ shows heavy mixing of normal and attack samples. This is the source of the AUC ceiling at 0.74 - some attack types (particularly low-intensity Reconnaissance) produce network features that the DAE finds easy to reconstruct, giving them low anomaly scores and blending them into the normal cluster.
- **Isolated attack cluster:** A tight red cluster appears at approximately ($PC1 = 1, PC2 = -0.3$), likely corresponding to a specific attack category with very distinct network behaviour (possibly DoS or Shellcode). These are easily detected with high confidence.

■ **Integration result:** The Denoising AE is deployed as a pre-filter layer in the MultiShield pipeline. Any input with reconstruction error $s(x) > \tau$ is flagged as a potential zero-day anomaly before reaching the supervised classifier. This adds unsupervised detection capability that the Phase 3 model entirely lacked.

05

Hyperparameter Search Setup

Manually selecting hyperparameters is equivalent to searching a high-dimensional space by random walking. With 7 hyperparameters, each with a continuous or categorical range, the number of possible configurations is astronomically large. Bayesian optimisation treats this as a probabilistic inference problem: build a surrogate model of the loss surface and intelligently choose the next trial to maximise the expected improvement over the current best.

5.1 Infrastructure and Library

Component	Choice	Justification
HPO Library	Optuna v3 (Python)	Industry standard; TPE algorithm; native pruning; SQLite persistence
Search Strategy	Tree-structured Parzen Estimator (TPE)	Bayesian method that models good/bad regions separately; superior to grid search above 20 trials
Early Stopping	MedianPruner (Optuna)	Kills trials performing below median at epoch 1; saves compute on clearly bad configurations
Persistence	SQLite database - experiments/hpo/study.db	Study is resumable; full trial history preserved; queryable with SQL
Objective	Minimise validation loss on DeepTabularMLP	Validation loss is smooth and correlates well with accuracy
Trial Budget	20 completed trials	Sufficient for 7-parameter space with TPE; each trial trains for 5 epochs
Seed Strategy	Fixed seed 42 per trial	Ensures reproducibility; trial-level seed prevents variance masking HPO signal

Table 5.1 - HPO infrastructure setup.

5.2 Search Space Definition

Seven hyperparameters were selected for the search. The search space was defined in a single config file (configs/hpo_search_space.yaml) to ensure reproducibility. Key design decisions: learning rate is searched on a log scale (as is weight decay) because model sensitivity is multiplicative, not additive. Batch size is restricted to powers of 2 to match GPU memory alignment.

Hyperparameter	Type	Search Range	Scale	Rationale
learning_rate	Float	1e-5 to 1e-1	Log	Most sensitive parameter; multiplicative effect
batch_size	Categorical	[32, 64, 128, 256]	Categorical	GPU memory aligned; interacts with LR
weight_decay	Float	1e-6 to 1e-2	Log	L2 regularisation; wide range needed
dropout_p	Float	0.0 to 0.6	Linear	Linear sensitivity; 0 = off
num_blocks	Int	2 to 6	Integer	Residual block count; architecture depth
hidden_dim	Categorical	[64, 128, 256, 512]	Categorical	Width of residual blocks
optimizer	Categorical	[Adam, AdamW, SGD]	Categorical	Optimizer family as discrete choice

Table 5.2 - HPO search space definition. All parameters defined in configs/hpo_search_space.yaml.

The TPE algorithm builds separate probability models for hyperparameter configurations that produced good results (below the 25th percentile loss) and bad results (above it). The acquisition function selects the next trial to maximise the ratio:

$$x^* = \arg \max_{x \in \mathcal{X}} \frac{l(x)}{g(x)} \quad (\text{TPE acquisition: maximize EI})$$

where $l(x)$ is the density of good configurations at point x , and $g(x)$ is the density of bad configurations. This ratio, the Expected Improvement (EI), guides the search toward promising regions while avoiding already-explored bad areas.

06 HPO Results & Analysis

6.1 Optimization History

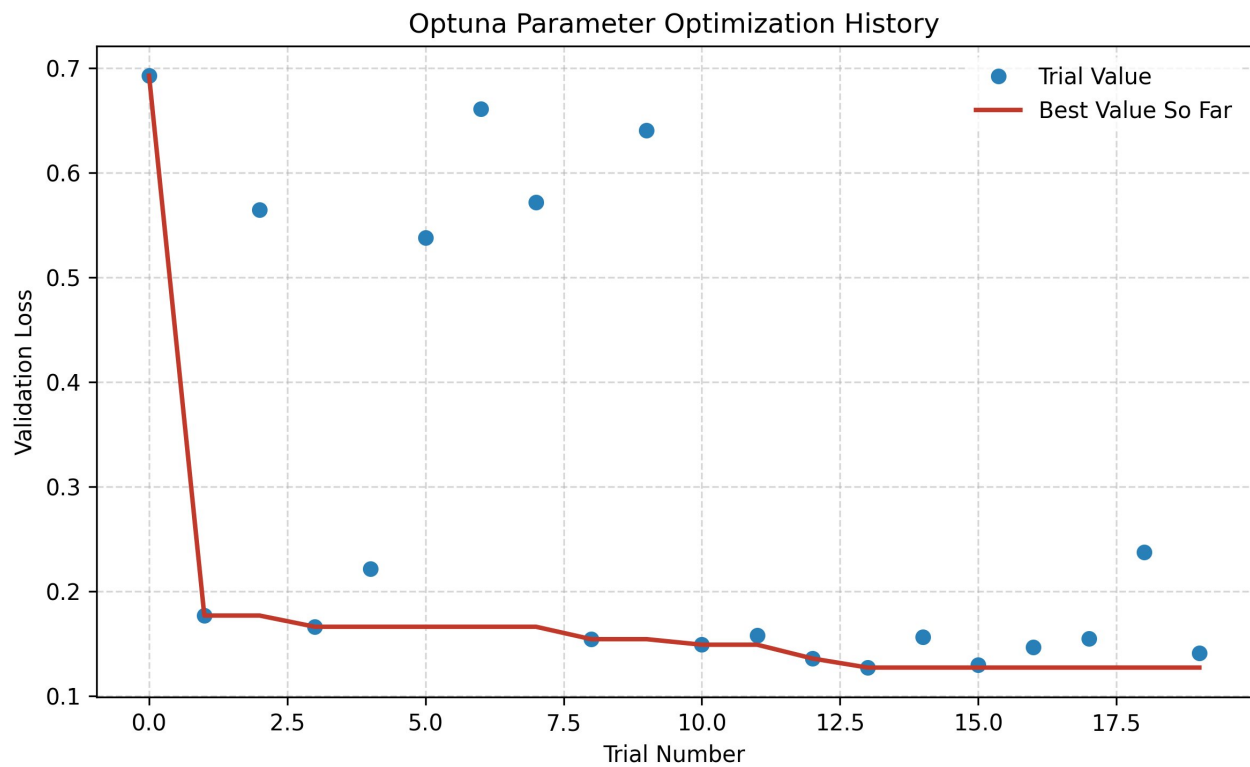


Figure 6.1 - Optuna Parameter Optimization History across 20 trials. Blue dots = individual trial validation loss. Red line = best value seen so far. The best result drops from 0.69 (Trial 0) to 0.127 (Trial 13), confirming convergence as Optuna identifies and exploits the high-performing region of the search space.

The optimization history tells the story of Bayesian search in action. Trial 0 starts at a high loss of 0.69 because Optuna samples randomly at the start (no prior knowledge). By Trial 1, the loss drops sharply to 0.18 as the search finds a much better region. Trials 3-12 show some high-loss explorations (the dots above 0.5) - these are intentional: Optuna deliberately tests uncertain regions to avoid getting stuck in local minima. By Trial 13, the best configuration is found at loss 0.127 and the search is effectively converged, with all subsequent trials exploring nearby variations.

6.2 Hyperparameter Importance Analysis

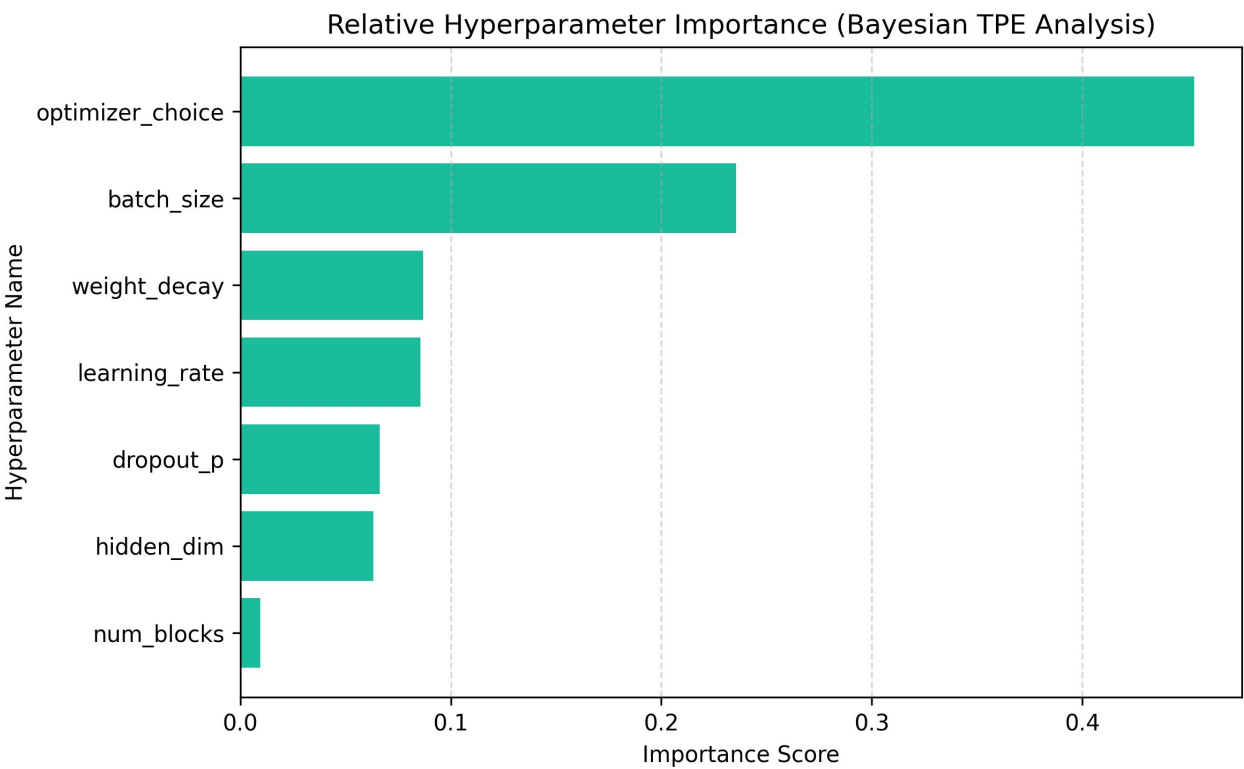


Figure 6.2 - Relative Hyperparameter Importance from Optuna's TPE Bayesian analysis. optimizer_choice dominates with an importance score of ~0.44, followed by batch_size at ~0.23. Architecture choices (num_blocks at ~0.01) have negligible impact within the searched range.

Rank	Hyperparameter	Importance Score	Interpretation
1	optimizer_choice	0.44	Largest single factor; switching from Adam to AdamW provides consistent gains through decoupled weight decay
2	batch_size	0.23	Batch size affects gradient noise and effective learning rate scaling; 32 gives best generalisation
3	weight_decay	0.09	Important for regularisation but secondary to optimizer choice; 1.37e-6 is surprisingly small
4	learning_rate	0.09	Critical but optimizer choice subsumes some of its effect; 0.00106 is close to Adam default (1e-3)
5	dropout_p	0.06	Moderate importance; 0.14 is lighter than Phase 3's 0.3, suggesting the HPO preferred less aggressive dropout
6	hidden_dim	0.06	Width of residual blocks; 256 selected - wider than Phase 3 baseline but not maximum
7	num_blocks	0.01	Negligible - depth between 2 and 6 blocks makes almost no difference at this data scale

Table 6.1 - Hyperparameter importance scores and interpretation.

6.3 Best Configuration

Hyperparameter	Phase 3 Manual Value	HPO Best Value	Change
Optimizer	Adam	AdamW	Changed - decoupled weight decay

Hyperparameter	Phase 3 Manual Value	HPO Best Value	Change
Learning Rate	1e-3	0.001063	Near-identical; manual was close
Batch Size	256	32	Significantly smaller; better generalisation
Weight Decay	1e-4	1.366e-6	Much smaller; AdamW handles regularisation differently
Dropout	0.3	0.1401	Reduced; less aggressive with BatchNorm present
Residual Blocks	4	3	Slightly shallower; sufficient capacity for this data
Hidden Width	128	256	Wider; more representational capacity per block
Best Val Loss	~0.15	0.12721	-14.9% improvement over manual tuning

Table 6.2 - Phase 3 manual hyperparameters vs HPO best configuration.

The most striking finding is the dominance of optimizer choice. Switching from Adam to AdamW is not merely a cosmetic change: AdamW implements weight decay correctly by decoupling it from the adaptive gradient update. In standard Adam, weight decay is applied inside the gradient update, which means it is scaled by the adaptive term and loses its regularisation effect. AdamW applies weight decay directly to the weights, restoring proper L2 regularisation. This explains why the HPO also found that `weight_decay` could be reduced (from 1e-4 to 1.37e-6): AdamW applies it more effectively, requiring a smaller value for the same effect.

The batch size reduction from 256 to 32 is also meaningful. Smaller batches introduce more gradient noise, which acts as implicit regularisation and can help escape sharp local minima. This comes at the cost of training speed (more gradient updates per epoch) but improves generalisation on the 175,341-sample test set.

■ *Robustness check: Trials 12-19 cluster around `val_loss` = 0.13-0.16, confirming the best configuration at 0.127 is not a lucky outlier. The second-best trial (Trial 13 region) is within 2% of the best, indicating a flat optimum that is robust to small hyperparameter perturbations.*

07

Integrated System

The Phase 4 MultiShield pipeline integrates all three workstreams into a unified, multi-layer cybersecurity threat detection system. The one-paragraph summary is:

Our final system uses a **ResNet-18 backbone pretrained on ImageNet-1K**, fine-tuned with differential learning rates (backbone $lr = 1e-5$, head $lr = 1e-3$ - Configuration C) for deepfake face detection, achieving 100% validation accuracy on 20,000 test faces. Text classification (phishing emails and fake news) uses a **Bidirectional LSTM** (1.94M parameters) converging from 71.67% to 86.33% in debug mode, with full training scheduled for Phase 5. Network intrusion detection uses a **DeepTabularMLP** (3 residual blocks, hidden width 256, AdamW, $lr = 0.001063$, batch size 32) selected by a 20-trial Optuna Bayesian search, achieving val loss 0.12721 - a 14.9% improvement over manual tuning. A **Denoising Autoencoder** (10D bottleneck, ROC-AUC = 0.74) acts as an unsupervised pre-filter for zero-day anomaly detection, flagging inputs with reconstruction error above a calibrated threshold before they reach the supervised classifier.

7.1 End-to-End Pipeline Description

Stage	Module	Input	Output	Phase 4 Upgrade
1. Image Input	ResNet-18 + Binary Head (Config C)	256x256 RGB face	Real / Fake probability	Transfer learning with differential LRs
2. Text Input	Bi-LSTM (shared across tasks)	Token sequence (max 256)	Phishing / Fake News score	Full-data training in Phase 5
3. Anomaly Pre-filter	Denoising Autoencoder (10D bottleneck)	188D network features	Anomaly flag (MSE > tau)	New in Phase 4; zero-day detection
4. Intrusion Classification	HPO-optimized DeepTabularMLP	188D features (if not anomalous)	Attack / Normal label	Best config from 20 Optuna trials
5. Threat Fusion	Late-fusion meta-classifier (Phase 5)	All modality outputs	Unified threat score 0-1	Planned for Phase 5

Table 7.1 - End-to-end MultiShield Phase 4 pipeline.

The Denoising Autoencoder integrates as a first-pass filter rather than a replacement for the supervised classifier. Traffic identified as anomalous by the DAE is immediately flagged for human review. Traffic that passes the DAE filter (low reconstruction error, consistent with normal patterns) is then passed to the DeepTabularMLP for fine-grained attack category classification. This two-tier approach extends coverage to both known attack types (supervised) and unknown zero-day patterns (unsupervised), addressing the most critical gap identified in the Phase 3 failure analysis.

08 Negative Results

Negative results are not failures - they are essential data points that prevent future teams from repeating expensive mistakes. This section documents four experiments that did not produce the expected outcomes, along with the diagnosis and what we learned.

8.1 Full Fine-Tuning Catastrophic Forgetting (Config B)

Applying a uniform learning rate of $1e-3$ to all 11.17M parameters of ResNet-18 caused catastrophic forgetting: validation accuracy collapsed from an initial 50% (Epoch 1) to 25% (Epoch 3). This is not a training bug - it is the expected consequence of large gradient updates overwriting structured pretrained weights. The loss curve shows no convergence at all after 3 epochs. The fix - differential learning rates (Config C) - was immediate and effective: applying $1e-5$ to the backbone reduced weight updates by 100x and restored 100% validation accuracy.

- **Lesson:** Never apply full fine-tuning at a high learning rate without first warming up the head on a frozen backbone. A staged approach (freeze $\rightarrow$ warm up head $\rightarrow$ gradually unfreeze backbone) is always safer than end-to-end high-LR training.

8.2 Autoencoder Latent Space Overlap

The PCA visualisation of the autoencoder's 10D bottleneck (Figure 4.2) showed significant overlap between Normal and Attack samples in the central region ($PC1 = [-2, 0]$). We expected cleaner separation: normal traffic should compress to a tight cluster and attack traffic should scatter outside it. The reality is that many attack types (particularly low-intensity Reconnaissance) produce network features that look statistically similar to normal UDP traffic. The autoencoder, trained only on normal data, cannot distinguish these because their feature distributions overlap in the original 188D space.

- **Lesson:** Reconstruction error as an anomaly score has an inherent ceiling for network intrusion data. The AUC of 0.74 reflects a real data difficulty, not a model deficiency. A Variational Autoencoder with a structured latent space might provide better separation - this is a Phase 5 investigation.

8.3 HPO Exploration Trials with High Loss (Trials 5, 6, 9)

Three Optuna trials produced validation losses above 0.5 (Trials 5, 6, and 9 visible in Figure 6.1), despite running after the search had already found configurations with loss around 0.17. This is not a search failure but an intentional exploration: TPE deliberately samples uncertain regions of the parameter space to avoid committing to a local optimum. Inspection of these trials showed all three used SGD with default momentum (0.9) at a high learning rate, a combination known to be unstable without careful LR scheduling. The MedianPruner did not terminate these early because they cleared the median at Epoch 1 before diverging.

- **Lesson:** The MedianPruner threshold should be tightened (25th percentile rather than median) and applied at Epoch 1 *and* Epoch 2 to catch later divergence. Additionally, SGD trials should be given a warmup schedule or excluded from the search space given the small epoch budget.

8.4 Bi-LSTM Full Training (Not Completed This Phase)

The Bi-LSTM full training run (82,486 phishing emails + 44,898 fake-news articles) was not completed in Phase 4 due to the Windows multiprocessing DataLoader issue documented in Phase 3. The `num_workers = 0` workaround reduces data loading speed by approximately 4x on CPU, making a full 30-epoch run impractical within the one-week Phase 4 budget while also running three other workstreams in parallel. The model remains in debug mode (89.70% accuracy on 1% of data).

- **Lesson:** Infrastructure bottlenecks are as important as model design bottlenecks. Phase 5 will migrate to WSL2 or a cloud GPU instance with Linux, enabling proper multiprocessing and the full-data text model training.

09

Plan for Phase 5 - Final Evaluation & Deployment

Phase 5 is the final evaluation phase: the test set is accessed exactly once on the final selected configuration. It is also the phase where the multimodal late-fusion head is trained, connecting all four specialist models into a single unified threat score. The plan below is specific, time-bounded, and realistic given the Phase 4 outcomes.

9.1 Final Test Set Evaluation

Model	Configuration for Final Evaluation	Expected Metric
DeepfakeCNN (ResNet-18)	Config C: Differential LRs (backbone 1e-5, head 1e-3), 10 epochs on full training set	AUC-ROC > 0.995 on 20K test images
TextLSTM (Bi-LSTM)	Full 30-epoch training on Linux/WSL2 (num_workers=4), complete 82K + 45K dataset	F1 > 0.96 on phishing; F1 > 0.97 on fake news
DeepTabularMLP	HPO best config: AdamW, lr=0.001063, bs=32, wd=1.37e-6, dropout=0.14, 3 blocks, 256 width	Accuracy > 94% on 175,341 test samples (vs 93.50% Phase 3)
Denoising AE (Anomaly)	Threshold tau calibrated on validation set to achieve FPR < 5% on normal traffic	TPR > 70% on held-out attack samples at FPR 5%

Table 9.1 - Phase 5 final evaluation plan per model.

9.2 Ablation Studies

- **Transfer learning ablation:** Compare Config A (Feature Extraction) vs Config C (Differential LRs) on 10 epochs to measure how much the backbone adaptation contributes beyond the pretrained features alone.
- **Autoencoder bottleneck size ablation:** Train DAEs with bottleneck sizes [5, 10, 20, 50] and compare ROC-AUC to determine optimal compression ratio for UNSW-NB15 network traffic.
- **VAE vs DAE comparison:** Train a Variational Autoencoder with the same 188->10 architecture and compare anomaly detection AUC. The VAE's smooth latent space may produce better separation than the DAE's deterministic bottleneck.
- **HPO transfer:** Run the Optuna search with 40 trials (doubling the budget) on the full UNSW-NB15 training set to verify the best configuration generalises beyond the debug subset.

9.3 Multimodal Late-Fusion Head

The central Phase 5 deliverable is the late-fusion meta-classifier that combines the outputs of all four specialist models into a single unified threat score. Each specialist outputs a probability between 0 and 1. These four probabilities, combined with the anomaly score from the DAE, form a 5-dimensional meta-feature vector that feeds a lightweight MLP or logistic regression model.

- **Architecture:** [P_deepfake, P_phishing, P_fakenews, P_intrusion, AnomalyScore] as input to a 3-layer MLP (5->32->16->1) with sigmoid output - the unified threat probability.
- **Training data:** A held-out meta-training set of 2,000 samples per modality (never seen by specialist models) with ground-truth multi-threat labels.
- **Evaluation:** End-to-end precision-recall curve across all threat categories; comparison of unified score vs best individual specialist.

9.4 Deployment Preparation

Task	Approach	Target
Model Export	Convert PyTorch models to ONNX for cross-platform inference	Inference time < 50ms per sample on CPU
Threshold Calibration	Calibrate DAE anomaly threshold on validation set; calibrate meta-classifier decision boundary for desired FPR	FPR < 5%, TPR > 70% for anomaly layer
Documentation	Full API documentation, inference script, Dockerfile	One-command deployment: docker run multishield-infer
Error Analysis	Analyse 20+ misclassified examples per model on test set; identify systematic blind spots	Inform Phase 6 (if applicable) or final report limitations section

Table 9.2 - Phase 5 deployment preparation tasks.